ListedWheels: Architecture and Build Guide

Version 1.0 · For Avinash and future maintainers · Read the *Architecture Document* first; this guide is the order of work.

1. How to read this guide

This guide records **how an AI assistant builds ListedWheels from nothing to launch**, in the order the work should happen. Each phase has the same five parts:

Part	Meaning
Goal	What exists at the end of the phase
Prompt to the AI	The kind of instruction given, so the process can be repeated
What gets built	Files and behaviour
Check it works	A test to run before moving on
If it breaks	The usual errors and fixes

Golden rules for building with AI

1. **One phase at a time.** Never ask for the whole product in one prompt. Small steps are easier to check and fix.
2. **Give the AI the architecture document** at the start of every session. It is the memory.
3. **Run everything.** Code the AI writes is a draft until tests pass and you have clicked through it.
4. **Commit to Git after each working step**, so any bad change can be undone.
5. **When an error appears, paste the full error text, the file name, and what you were doing.** Never paraphrase an error.
6. **Money, auth and documents are reviewed by a human** (Avinash or the owner) before going live, even when the tests pass.

2. Phase map

Phase	Topic	Result
0	Tools and project setup	Empty project that runs

Phase	Topic	Result
1	Design system	Fonts, colours, texture, components
2	Database	Tables and migrations
3	Backend foundation	App, config, errors, health check
4	Google login	Sign-in and roles
5	Listings and photos	Create, edit, search
6	Verification and admin	Approve and reject
7	Pricing, promo codes, payments, invoices	Paid and free listings
8	Reviews, reports, enforcement	Trust features
9	Frontend pages	Every screen, mobile first
10	SEO, safety, polish	Search visibility and hardening
11	Deploy to AWS	Live site
12	Launch and operate	Monitoring and routine

3. Phase 0: Tools and project setup

Goal: a project on your machine that starts with one command.

Install: Python 3.12, Node 20, PostgreSQL 16 (or Docker), Git, VS Code, and an AI coding assistant.

Prompt to the AI: "Using the attached architecture document, create the backend folder structure exactly as listed, with FastAPI, a `pyproject.toml`, a `.env.example`, and a `/health` endpoint. Then create the Next.js frontend with TypeScript and Tailwind."

What gets built: the folder tree from the architecture document, a `docker-compose.yml` that runs Postgres locally, `.env.example` listing every setting (database URL, Google client ID, Razorpay keys, S3 bucket, JWT secret), and a `.gitignore` that excludes `.env`.

Check it works

```
cd backend && uvicorn app.main:app --reload
# open http://localhost:8000/health  -> {"status":"ok"}
# open http://localhost:8000/docs    -> interactive API page
```

If it breaks

- `ModuleNotFoundError`: the virtual environment is not active, or a package is not installed. Activate it and run the install command again.
- *Port already in use*: another program holds port 8000. Stop it or choose another port.
- *Cannot connect to database*: Postgres is not running, or the URL in `.env` has the wrong password or port.

4. Phase 1: Design system (UI and UX)

This phase comes before any page, so every screen looks like one product. The goal is a **trustworthy, clean, premium** feel: buyers are about to spend a large amount of money, so the interface must feel calm and credible.

4.1 Design principles

1. **Trust first.** Verified badges, clear prices, no clutter, no pop-up ads.
2. **Photos lead.** Vehicles sell through pictures, so images get the most space.
3. **One action per screen.** Each page has a single obvious primary button.
4. **Mobile first.** Design for a 360 px phone, then widen.
5. **Fast.** Small images, little animation, no layout jumps.

4.2 Colour palette

A deep navy for trust, a warm saffron accent for action (it reads well in India and stands out from the usual blue of other portals), and warm off-white backgrounds to avoid a sterile look.

Token	Hex	Use
`ink-900`	`#0F1B2D`	Headings, primary text, dark surfaces
`ink-700`	`#2A3A52`	Body text
`ink-500`	`#5B6B82`	Secondary text
`ink-200`	`#D9DEE6`	Borders and dividers
`paper`	`#FAF8F4`	Page background (warm off-white)
`surface`	`#FFFFFF`	Cards
`accent-500`	`#F5A524`	Primary buttons, highlights
`accent-600`	`#D98A0B`	Button hover
`success`	`#1F9D6B`	Verified, live, sold-confirmed

Token	Hex	Use
`warning`	`#C98A00`	Pending, expiring
`danger`	`#C0392B`	Errors, removal, report

Contrast rule: body text must reach at least 4.5:1 against its background (WCAG AA). Navy on off-white passes easily. On the saffron button, use navy text (`ink-900`), **not** white, because white on saffron fails contrast.

4.3 Typography

Role	Font	Weights	Why
Headings and prices	**Sora** (Google Fonts)	600, 700	Modern, geometric, confident
Body and UI	**Inter**	400, 500, 600	Very legible on small screens, has tabular numerals
Numbers in tables and invoices	Inter with `font-variant-numeric: tabular-nums`		Digits line up

Type scale (mobile / desktop)

Style	Mobile	Desktop	Line height
Display	32 px	48 px	1.15
H1	26 px	36 px	1.2
H2	21 px	28 px	1.25
H3	18 px	22 px	1.3
Body	16 px	16 px	1.6
Small	14 px	14 px	1.5
Price	24 px	30 px	1.1

Rules: never go below 14 px for text. Keep line length near 60 to 75 characters. Indian-rupee prices use the Indian grouping, for example `₹1,99,999`, produced with `Intl.NumberFormat('en-IN')`.

4.4 Texture and depth

"Texture" means subtle surface quality so the site does not feel flat.

- **Paper grain:** a tiny noise image (a 200 px PNG or SVG filter) laid over the page background at 3 to 4 percent opacity. It gives a printed-paper warmth. It must be nearly invisible.
- **Soft shadows:** cards use a two-layer shadow, for example `0 1px 2px rgba(15,27,45,.06), 0 8px 24px rgba(15,27,45,.08)`. Avoid hard black shadows.
- **Rounded corners:** 12 px on cards, 10 px on inputs and buttons, full round on badges.
- **Dark hero:** the home banner uses `ink-900` with a faint diagonal line pattern at 5 percent opacity, so the saffron button stands out.
- **Photo treatment:** a very light inner border and a 4 percent dark gradient at the bottom of photos so overlaid price text stays readable.

4.5 Spacing, grid and breakpoints

- **Spacing scale (8-point):** 4, 8, 12, 16, 24, 32, 48, 64, 96 px.
- **Breakpoints:** 360 (phone), 640 (large phone), 768 (tablet), 1024 (laptop), 1280 (desktop).
- **Listing grid:** 1 column on phones, 2 on tablets, 3 on laptops, 4 on wide screens.
- **Content width:** maximum 1200 px, centred.
- **Touch targets:** at least 44 by 44 px for every button and link.

4.6 Core components

Build each once in `components/ui`, then reuse it.

Component	Key behaviour
Button	Primary (saffron), Secondary (outline navy), Ghost, Danger; loading state; disabled state
Input and Select	Visible label above, helper text below, red border and message on error
Card	White surface, 12 px radius, soft shadow, lifts 2 px on hover (desktop only)
Badge	Verified (green), Pending (amber), Sold (grey), Featured (saffron)
ListingCard	4:3 photo, price in Sora, title, year and km and fuel chips, city, verified badge

Component	Key behaviour
FilterBar	Horizontal chips on mobile that open a bottom sheet; sidebar on desktop
PriceBreakdown	Base fee, discount, GST, total in a tidy table with the total in bold
Modal and BottomSheet	Bottom sheet on phones, centred dialog on desktop; closes on Escape and outside tap
Toast	Short success and error messages, auto-hide in 4 seconds
Skeleton	Grey placeholder shapes while data loads, so the page does not jump
EmptyState	Friendly message and one action when a list has no results

4.7 Motion

- Durations 150 to 250 ms with an ease-out curve.
- Animate only opacity and transform, which are cheap.
- Respect `prefers-reduced-motion` by turning animation off.

4.8 Accessibility checklist

- Every image has meaningful `alt` text; every form field has a label.
- All actions work with a keyboard, and a visible focus ring (2 px saffron outline) is shown.
- Colour is never the only signal; badges also carry text.
- Test with a screen reader once before launch.

4.9 Implementation

Prompt to the AI: "Create a Tailwind configuration using these exact colour tokens, fonts and spacing. Create the core components listed, each in its own file, with all states. Create a `/styleguide` page that shows every component so we can review the design in one place."

What gets built: `tailwind.config.ts` with the tokens, a global CSS file with the grain texture and font loading (`next/font` for Sora and Inter, which avoids layout jumps), the component files, and the style guide page.

Check it works: open `/styleguide` on a phone-width browser window (360 px) and on a laptop. Everything should align, and nothing should overflow sideways.

If it breaks

- *Fonts flash or jump:* load them through `next/font`, not a plain stylesheet link.
- *A colour does not apply:* the class name is misspelt, or the token is missing from the Tailwind config, so Tailwind silently ignores it.
- *Horizontal scrolling on phones:* an element has a fixed width wider than the screen. Use `max-w-full` and `overflow-x-auto` for tables.

5. Phase 2: Database

Goal: every table from the architecture document exists, with correct constraints.

Prompt to the AI: "Write Alembic migrations for the tables in section 7 of the architecture document. Include primary keys, foreign keys, NOT NULL rules, status check constraints, the partial unique index on `reg_no_hash`, and the search indexes. Store money as integer paise."

What gets built: one migration per table group, in dependency order (users, then seller profiles and documents, then listings and media, then plans, orders, payments, invoices, then promos, reviews, reports, audit and bans), plus a seed script inserting the four plans.

Check it works

```
alembic upgrade head      # creates all tables
alembic downgrade -1      # undo works
alembic upgrade head
```

Then try to insert two live listings with the same registration hash. The second must be rejected by the database.

If it breaks

- *"relation does not exist":* a migration has not run, or ran in the wrong order.
- *"multiple heads" in Alembic:* two migrations branch from the same parent. Merge them with `alembic merge heads`.
- *Foreign key error on insert:* the parent row is missing. Create it first.

6. Phase 3: Backend foundation

Goal: the layered skeleton is in place and every layer follows its rules.

Prompt to the AI: "Create `core/config.py` using Pydantic Settings, `core/errors.py` with domain error classes and one handler that returns `{code, message}`, structured logging with a request ID, a database session dependency, and one complete example slice (route, schema, service, repository) for a simple resource, so we can copy the pattern."

What gets built: the pattern every later feature copies. The example slice is the template: route calls service, service calls repository, repository runs SQL.

Check it works: call the example endpoint, confirm the response, force an error and confirm the JSON error shape and the log line with a request ID.

If it breaks

- *Pydantic "field required":* the request body is missing a field, or the schema names a field differently from the JSON.
- *Pydantic "value is not a valid ...":* the type is wrong. Read the field path in the error to find which one.
- *"coroutine was never awaited":* an `async` function was called without `await`.
- *Business logic found in a route:* move it into the service. This is the most common drift when AI writes code, so check for it in every review.

7. Phase 4: Google login and roles

Goal: a user signs in with Google and the API knows who they are.

Setup by hand (once): in Google Cloud Console create an OAuth client ID, add the local and production site URLs as authorised origins, and copy the client ID into `.env`.

Prompt to the AI: "Implement `POST /auth/google` that verifies a Google ID token (signature, audience, expiry), finds or creates the user, and returns a 15-minute access token plus an HttpOnly refresh cookie. Add `/auth/refresh`, `/auth/logout`, `GET /me`, `current_user` and `require_admin` dependencies, and Terms of Service consent storage."

Check it works: sign in from the frontend, call `/me`, wait for the token to expire and confirm refresh works, and call an admin endpoint as a normal user (expect 403).

If it breaks

- *"redirect_uri_mismatch" or "origin not allowed":* the exact site URL is not listed in the Google console. Add it, including the port for local work.
- *Token verification fails with "audience":* the client ID in `.env` differs from the one used by the frontend.

- *Login works locally but not live:* cookies need `Secure` and the right `SameSite` on HTTPS, and CORS must list the real domain.
- *401 on every request:* the access token is not being sent, or the clock on the server is wrong, so the token looks expired.

8. Phase 5: Listings and photos

Goal: a seller can create a draft, add photos, and a buyer can search.

Prompt to the AI: "Implement the listing schemas with per-category rules (cycles have no mileage and no registration; cars and bikes require registration, fuel type and, for cars, transmission). Implement create, edit, mark-sold, the locked-field rule, photo upload with pre-signed S3 URLs, EXIF removal, and a search endpoint with filters, sorting and pagination. Hash and mask the registration number."

What gets built: Pydantic schemas using a discriminated union on category, the listing service (status transitions in one place), the search repository, and the media service. A background step strips EXIF and resizes photos into a few widths.

Check it works: create one listing per category, upload three photos, search with each filter, and confirm a second listing for the same registration is refused.

If it breaks

- *S3 upload gives "SignatureDoesNotMatch":* the content type used when signing differs from the one used in the upload.
- *S3 upload blocked in the browser:* the bucket's CORS rule does not allow the site's origin and the PUT method.
- *Search is slow:* the filter columns lack the composite index. Use `EXPLAIN ANALYZE` to see.
- *Status jumps to an invalid value:* the transition rules are not enforced in the service. Put them in one function.

9. Phase 6: Verification and admin

Goal: an admin can review documents and approve or reject listings.

Prompt to the AI: "Implement document upload to a private bucket with server-side encryption, an admin queue endpoint, approve and reject (reject requires a reason), hide, remove and ban, and write every admin action to `audit_log`. Documents are served only through short-lived signed URLs to admins."

Check it works: approve a listing and confirm `expires_at` is 365 days ahead; reject one and confirm the reason is stored; open a document URL after it expires (it must fail).

If it breaks

- *Admin sees "Access denied":* the user's role is not set to admin in the database.
- *Audit log is empty:* the action wrote its change outside the transaction. Write both in the same transaction.

10. Phase 7: Pricing, promo codes, payments, invoices

This is the highest-risk phase. A human reviews it line by line.

Prompt to the AI: "Implement a pricing function in paise: base fee from the plan, minus promo discount, GST 18 percent on the discounted amount, rounded consistently. Implement promo code validation with rate limiting and a generic error, order creation, Razorpay order creation, a webhook that verifies the signature and is idempotent, the zero-total path that skips Razorpay, invoice generation with sequential numbers and the CGST/SGST/IGST split, credit notes, and a monthly GST CSV export."

Worked example (must match a unit test): a ₹1,999 car listing with a ₹200 promo has a discounted base of ₹1,799, GST of ₹323.82, and a total of ₹2,122.82, which is `212282` paise.

Check it works

1. Unit tests for the pricing function with several promo cases, including 100 percent.
2. Pay with Razorpay **test** keys and confirm the order, payment, invoice and listing status all update.
3. Send the same webhook twice and confirm nothing is duplicated.
4. Send a webhook with a wrong signature and confirm it is rejected.
5. Use a 100 percent code and confirm a zero-value invoice and a redemption record exist.

If it breaks

- *Webhook signature fails:* the raw request body must be used for the signature check, not a re-encoded copy; the webhook secret must match the dashboard.
- *Payment succeeded but the listing did not update:* the browser result was trusted instead of the webhook, or the webhook URL is unreachable. Use a tunnelling tool for local tests.
- *Totals off by a paisa:* floats were used somewhere. Use integers and one rounding rule.
- *Duplicate invoices:* the webhook handler is not idempotent. Key it on the Razorpay payment ID.

11. Phase 8: Reviews, reports, enforcement

Prompt to the AI: "Implement reviews allowed only after a recorded WhatsApp contact, one per buyer per seller; reports with reasons; an automatic flag (not an automatic ban) on many negative reviews or reports in a short time; and the enforcement ladder with bans writing hashes to the `bans` table so re-registration is blocked."

Check it works: a buyer with no contact cannot review; a banned seller cannot sign up again with the same registration or document hash.

If it breaks: a review is refused unexpectedly, which usually means the contact event was never recorded. Confirm the contact endpoint writes to `contact_events`.

12. Phase 9: Frontend pages (mobile first)

Build in this order, testing each on a 360 px screen first.

Order	Page	Notes
1	Home	Dark hero, category tiles (Cars, Bikes, Cycles), search box, recent listings, trust strip (Verified sellers, Safe tips)
2	Search results	Filter chips and bottom sheet on mobile, sidebar on desktop, sort, skeleton loading, empty state
3	Listing detail	Photo carousel with swipe, price in Sora, key facts grid, masked registration, Verified badge, WhatsApp button (sticky at the bottom on phones), Report link, safety banner
4	Sign-in	Single Google button and the Terms consent text
5	Create listing	Step form: category, details, photos, documents, video link, review, pay
6	Checkout	Price breakdown, promo field, Terms checkbox, Razorpay
7	Seller dashboard	Listings with status badges, expiry countdown, renew, mark sold, views and WhatsApp clicks, invoices
8	Dealer profile	Shop details, Verified badge, listings, reviews
9	Favorites	Saved listings

Order	Page	Notes
10	Admin	Queue, listing and user management, promo codes, reviews, reports, GST export
11	Legal	Terms, Privacy Policy, Refund Policy, Safety Tips, Grievance contact

UX details that matter

- **The create-listing form saves a draft after every step**, so a lost connection never loses work.
- Show the **full price breakdown before payment** (base, GST, total).
- Inline validation messages sit beside the field, in plain language ("Enter the year as four digits").
- The WhatsApp button opens a prefilled message such as "Hi, I saw your listing on ListedWheels: [title], ₹[price]".
- Photos are lazy-loaded with fixed aspect ratios so the layout never jumps.

Prompt to the AI: "Build the [page name] page using only the components from `components/ui` and the design tokens. It must work at 360 px first. Fetch data through the generated API client. Include loading, empty and error states."

If it breaks

- *Hydration mismatch warnings:* the server and the browser render different output, often from dates, random values or `window` access. Move those into a client-only effect.
- *CORS errors in the browser console:* the backend must list the frontend origin and allow credentials.
- *Stale data after an update:* the cached request was not refreshed. Invalidate it after a successful change.

13. Phase 10: SEO, safety and polish

SEO

- Each listing has a clean URL such as `/cars/chennai/2019-maruti-swift-vxi-1042`.
- Unique title and meta description per listing, Open Graph image (so WhatsApp and Instagram shares show a photo and price), `sitemap.xml`, `robots.txt`.
- Structured data (`Vehicle` and `Offer`) on listing pages.
- Sold and expired listings stay reachable with a notice for a few weeks, then return 410.

Safety

- Rate limits on login, contact, promo and listing creation.
- CAPTCHA on listing creation.
- Flag unrealistically low prices for admin review.
- Security headers (HSTS, content type, frame options) and a content security policy.
- Run a dependency vulnerability scan.

Performance targets: Largest Contentful Paint under 2.5 seconds on a mid-range phone on 4G; photos served as WebP at the correct width.

14. Phase 11: Deploy to AWS

1. Create the staging environment first and rehearse everything there.
2. Build the backend into a Docker image and push it to ECR.
3. Create the database, run migrations, and seed plans.
4. Create the S3 buckets: public photos behind CloudFront, private documents with encryption and no public access.
5. Store secrets in Secrets Manager or Parameter Store.
6. Point the domain at the site with HTTPS (ACM certificate).
7. Set the Razorpay webhook to the live URL and switch from test to live keys.
8. Add a Budgets alert at $5 and $20, and CloudWatch alarms for errors and for database storage.
9. Run the full checklist from section 16 on staging, then repeat on production.

If it breaks

- *Container starts then stops:* read the logs. Usually a missing environment variable.
- *Backend cannot reach the database:* the security group does not allow the backend to reach port 5432.
- *Images do not load:* the CloudFront origin or bucket policy is wrong.
- *Unexpected AWS bill:* look in Cost Explorer for the service. Common culprits are NAT gateways, large instances and data transfer.

15. Troubleshooting playbook (for future errors)

Use this order every time.

1. **Where did it fail?** Browser console, network tab, or server log. Find the request ID in the log.

2. **Which layer?** Route (wrong input or auth), service (wrong rule), repository (wrong SQL), or an outside service (Google, Razorpay, S3).
3. **Reproduce it** with the smallest possible request, using the `/docs` page.
4. **Write a failing test** that captures the bug, then fix it.
5. **Ask the AI with context:** paste the full error, the relevant file, what you expected, and what happened.
6. **Never fix by guessing in production.** Reproduce on staging first.

Symptom	Likely layer	First check
401	Route or auth	Token sent? Expired? Clock correct?
403	Auth	User role
404	Route	URL, router registered in `main.py`
422	Route (Pydantic)	Error path shows which field is wrong
500	Service or repository	Server log, stack trace, request ID
Wrong price or total	Service	Pricing unit tests
Slow search	Repository	`EXPLAIN ANALYZE`, indexes
Payment mismatch	Webhook	Signature, idempotency, raw body
Photos missing	Media or S3	CORS, bucket policy, signed URL expiry

16. Pre-launch checklist

Product: all listing categories work end to end; pricing and GST match hand calculations; a 100 percent promo code works and shows a zero invoice; refund and enforcement rules are in the Terms and shown at checkout.

Legal: Terms of Service, Privacy Policy, Refund Policy, grievance officer, and a data deletion route are live; a lawyer has reviewed the handling of Aadhaar and RC copies.

Security: secrets are not in Git; admin accounts use two-factor; buckets are private where they should be; rate limits are active; webhook signature is verified.

Quality: tests pass; staging was exercised on a real phone; screen-reader pass done; backups restore correctly (test a restore, not just the backup).

Operations: Sentry and CloudWatch alerts reach a real phone; Budgets alert is set; there is a written runbook for "site is down" and "payment problem".

17. Operating routine after launch

- **Daily:** review the verification queue and reports; check error alerts.
- **Weekly:** review promo code redemptions per dealer; check promo videos waiting for feedback.
- **Monthly:** export the GST CSV; reconcile Razorpay settlements; review AWS cost against the remaining credits.
- **Quarterly:** update dependencies; test a database restore; review the open product questions.

18. Glossary for Avinash

Term	Meaning
Route / endpoint	A URL the backend answers, such as `GET /listings`
Schema (Pydantic)	A definition of what data must look like; bad data is rejected automatically
Service	Code holding the business rules
Repository	Code holding the SQL
Migration	A versioned change to the database structure
JWT	A signed token proving who the user is
Webhook	A message Razorpay sends to our server when a payment changes
Idempotent	Safe to run twice with the same effect as once
Paise	One hundredth of a rupee; we store money in paise
EXIF	Hidden data in photos, including location, which we remove
CDN	A network that serves images quickly from a nearby location
SEO	Making pages easy for Google to find and rank

19. India-wide launch: what changes in the build

The site launches for all of India, so these additions go into the phases below. They match section 17 of the Architecture Document.

Phase	Addition	Check
1 Design system	Add **Noto Sans Devanagari** as a fallback font beside Inter and Sora, so Hindi renders cleanly later. Leave room for text that is 20 to 30 percent longer in other languages.	Switch a test page to Hindi text; buttons must not overflow.
2 Database	Migrations for `states` and `cities`, plus a seed script with all states and union territories and the major cities.	Every state and union territory is present; slugs are unique.
3 Foundation	Set up the translation library with an English file only, and read all visible text from it.	Changing one string in the file changes it everywhere.
5 Listings	`state_id` and `city_id` on listings; registration number normalised, validated by state code, then hashed.	`tn 09 ab 1234`, `TN09AB1234` and `TN-09-AB-1234` produce the same hash.
7 Payments	Place of supply from the seller's state. Same state as our registration gives CGST plus SGST; other states give IGST.	Unit tests with one same-state and one other-state seller; totals match by hand.
9 Frontend	A state and city picker in the header (bottom sheet on phones) that remembers the last choice, and a language switcher that shows English only for now.	Choose a city, reload, and the choice persists.
10 SEO	Location-based URLs and a sitemap per state.	Each state sitemap lists only that state's live listings.
11 Deploy	Create all AWS resources in `ap-south-1` (Mumbai).	The database, buckets and backend show the same region in the console.

If it breaks

- *A valid registration is rejected:* the validator is stricter than the real formats. Log the rejected values for a week and loosen the pattern.
- *Wrong GST type on an invoice:* the seller's state is missing or stale. Take it from the seller profile at order time and store it on the invoice.
- *Hindi text shows boxes:* the Devanagari font did not load. Check the font subset in `next/font`.
- *Slow pages for distant states:* images are not going through CloudFront, or the cache rule is missing.